



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 41

Dr. Asad Mahmood
May 11th, 2026

Lecture Outline

- Announcements (Quiz Solutions, Prefinals, Final Exam Preparations)
- Outline of Previous Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.4 Measuring and Improving Cache Performance
- Outline of Today's Lecture:
 - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
 - 5.4 Measuring and Improving Cache Performance

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.9 onwards (self-reading)

Average Access Time

- Hit time is **also important** for performance
 - Many possible reasons e.g. increase in cache size
- Average memory access time (AMAT)
- Example
 - Given:
 - CPU with 1ns clock,
 - hit time = 1 cycle,
 - miss penalty = 20 cycles,
 - I-cache miss rate = 5%
 - Find AMAT

Performance Summary

- When CPU performance increased **miss penalty becomes more significant**
 - E.g. Decreasing base CPI or increasing clock-rate results in greater proportion of time spent on memory stalls
- **Conclusion: Can't neglect cache behavior when evaluating system performance!**

Reducing Cache Misses via Flexible Block Placement

Associative Caches

■ Fully associative

- A given block can go in any cache index/entry/block/line
- Requires all entries to be searched at once
- Comparisons need to be done for all entries (expensive)

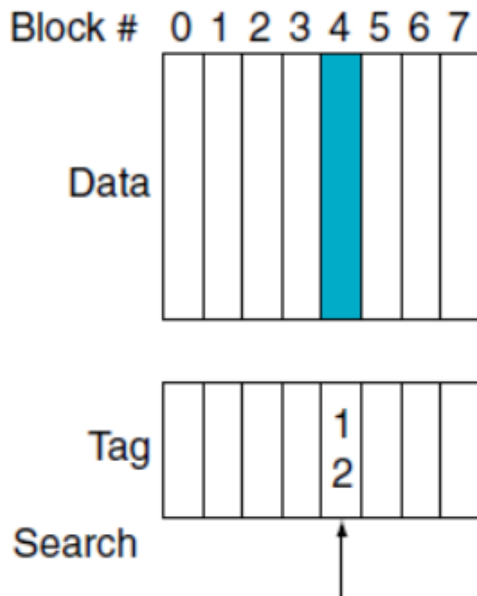
■ *n*-way set associative

- Cache divided into sets, with *n* entries/blocks per set
- A given block can go in any location/entry/block within a SET.
- Search all entries in a given set at once
- *n* comparators needed (less expensive)
- Set number can be determined from block address/number

Associative Cache Example

- Cache size = 8 blocks
- Desired block memory address = 12
- Cache location ?

Direct mapped



Set associative



Fully associative



Spectrum of Associativity

- We can also think of all block placement strategies as a variation on set associativity.
- Consider a 8-block cache
- One-way set-associative cache
- Two-way set-associative cache
- Four-way set-associative cache
- Eight-way set-associative cache

One-way set associative
(direct mapped)

Set	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Associativity Example

- Compare three small caches, each consisting of four one-word blocks
 - Direct mapped, 2-way set associative, fully associative
 - Block access sequence: 0, 8, 0, 6, 8
- Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

Associativity Example

■ 2-way set associative

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0	miss	Mem[0]			
8	0	miss	Mem[0]	Mem[8]		
0	0	hit	Mem[0]	Mem[8]		
6	0	miss	Mem[0]	Mem[6]		
8	0	miss	Mem[8]	Mem[6]		

■ Fully associative

Block address		Hit/miss	Cache content after access			
0		miss	Mem[0]			
8		miss	Mem[0]	Mem[8]		
0		hit	Mem[0]	Mem[8]		
6		miss	Mem[0]	Mem[8]	Mem[6]	
8		hit	Mem[0]	Mem[8]	Mem[6]	

Associativity Example

- If we had 8 blocks in the cache for previous example, the performance of 2-way associative and fully-associative would have been the same!
- If we had 16 blocks in the cache for previous example, the performance of all three placement strategies (direct-mapped, 2-way associative, fully-associative) would have been the same!
- Conclusion: Cache size and associativity are not independent in determining cache performance.

How Much Associativity

- Increased associativity decreases miss rate
 - But with diminishing returns
- Simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000

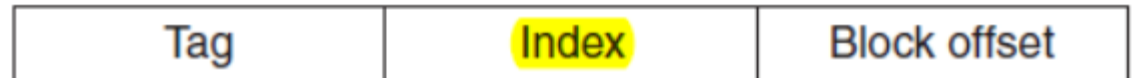
Associativity	Data miss rate
1	10.3%
2	8.6%
4	8.3%
8	8.1%

Locating a block in Set Associative Cache

- Like direct mapped cache, each block has a tag
- The tag of every cache block within the appropriate set is checked

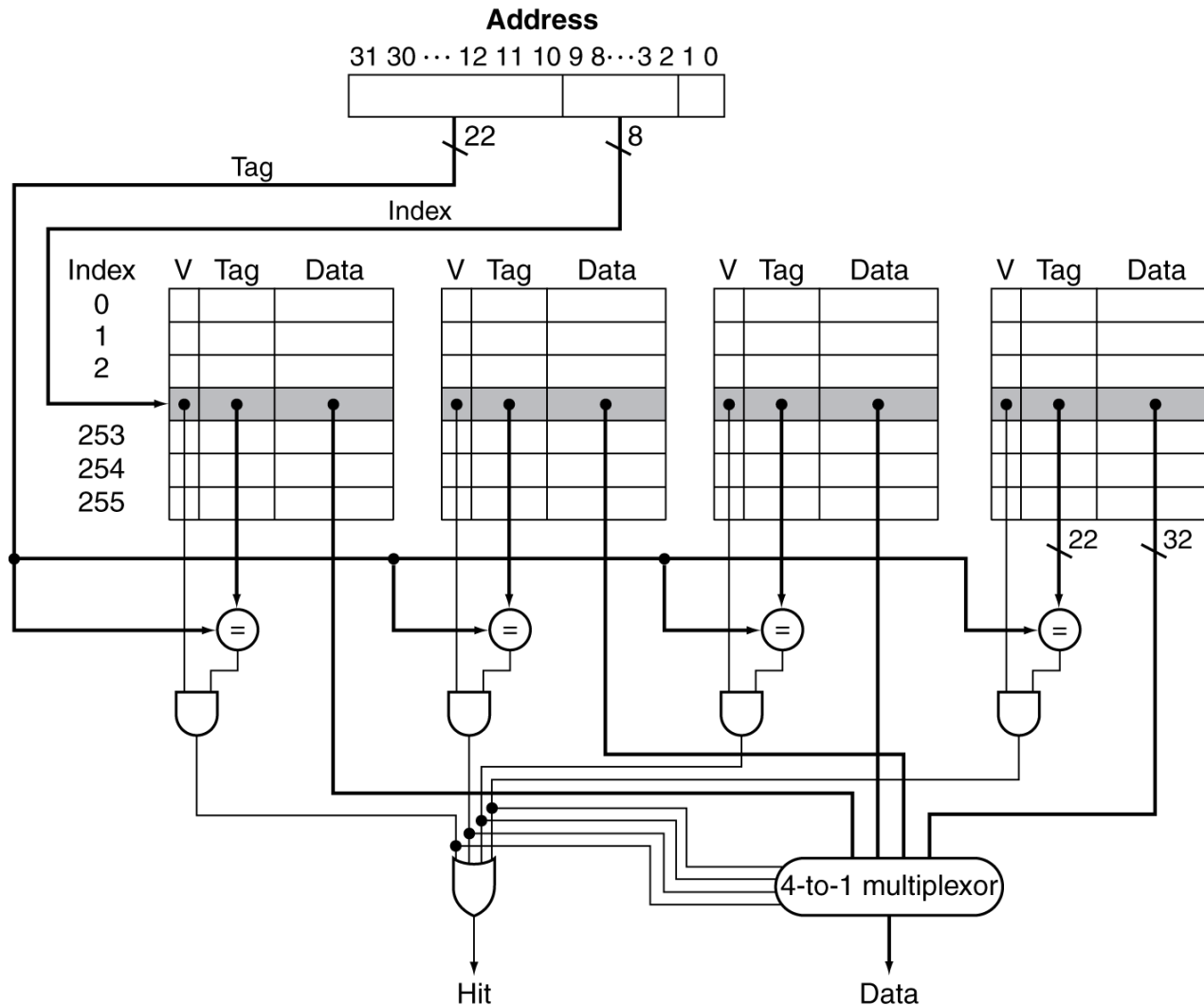
..... in parallel

- Three portions of the memory address



- Impact of increase in associativity on number of comparisons needed
- Impact of increase in associativity on number of Tag bits
 - E.g. a factor of 2 increase in associativity
 - Example

Set Associative Cache Organization



Replacement Policy

- Direct mapped: **no choice**
- Set associative
 - Prefer **non-valid entry**, if there is one
 - Otherwise, choose among entries in the set
- Least-recently used (LRU)
 - **Choose the one unused for the longest time**
 - Simple for 2-way, manageable for 4-way, **too hard beyond that**
- Random
 - Gives **approximately the same performance as LRU** for high associativity!

Multilevel Caches

- Primary cache attached to CPU
 - Small, but fast
- Level-2 cache services misses from primary cache
 - Larger, slower
- Main memory services L-2 cache misses
- Some high-end systems include L-3 cache

Multilevel Cache Example

- Given
 - CPU base CPI = 1, clock rate = 4GHz
 - Miss rate/instruction = 2%
 - Main memory access time = 100ns
- With just primary cache

Multilevel Cache Example

Now add L-2 cache

- Access time (for both hit and miss) = 5ns
- Reduces miss rate to main memory to 0.5%
- Penalty in case of Primary miss with L-2 hit?
- Penalty in case of Primary miss with L-2 miss?
- Effective CPI?
- Performance ratio?